

CS3P1. Computación Paralela y Distribuida (Obligatorio)

1. Información general

1.1 Escuela	: Computación
1.2 Curso	: CS3P1. Computación Paralela y Distribuida
1.3 Semestre	: 8° Semestre
1.4 Prerrequisitos	: CS212. Análisis y Diseño de Algoritmos. (5to Sem) CS231. Redes y Comunicación. (7mo Sem)
1.5 Condición	: Obligatorio
1.6 Modalidad de aprendizaje	: Presencial
1.7 Horas	: 2 HT; 4 HP
1.8 Créditos	: 3
1.9 Plan	: Plan Curricular 2016

2. Profesores

Alvaro Henry Mamani-Aliaga <ahmamani@ucsp.edu.pe>
- Doctor en Ciencia de la Computación, UNSA, Perú, 2019.
- Máster en Ciencia de la Computación, IME-USP, Brasil, 2011.

3. Fundamentación del curso

La última década ha sido testigo de un crecimiento explosivo en el campo de la computación con multiprocesadores, que incluye el desarrollo de procesadores de varios núcleos y la proliferación de centros de datos distribuidos. Como resultado, la computación paralela y distribuida ha pasado de ser un tema opcional a convertirse en uno de los componentes principales en los planes de estudio de ciencias de la computación a nivel de pregrado. Toda la computación paralela como la distribuida implican la ejecución simultánea de múltiples procesos, cuyas operaciones tienen el potencial de entrelazarse de manera compleja. Estos campos construyen sobre fundamentos en diversas áreas, como la concurrencia y la ejecución en paralelo, la consistencia y manipulación del estado de la memoria, y la tolerancia en los sistemas. La comunicación y coordinación entre los procesos se basa en el paso de mensajes y en modelos de memoria compartida, así como en conceptos algorítmicos como la atomicidad, el consenso y la espera condicional. Para lograr aceleración en la práctica, es necesario comprender algoritmos paralelos, estrategias de descomposición de problemas, arquitectura de sistemas, estrategias de implementación y análisis de rendimiento. Los sistemas distribuidos presentan desafíos adicionales en términos de seguridad y tolerancia a fallos, y requieren un énfasis en el mantenimiento del estado replicado, así como la resolución de problemas adicionales en el ámbito de las redes de computadoras.

4. Resumen

- | | |
|--|--|
| 1. Bases teóricas de la Computación Paralela y Distribuida | 2. Sistemas distribuidos |
| 3. Comunicación y coordinación | 4. Programación para el Procesamiento Masivamente Paralelo |

5. Objetivos Generales

- Que el alumno sea capaz de crear aplicaciones paralelas de mediana complejidad aprovechando eficientemente máquinas con múltiples núcleos.
- Que el alumno sea capaz de comparar aplicaciones secuenciales y paralelas.
- Que el alumno sea capaz de convertir, cuando la situación lo amerite, aplicaciones secuenciales a paralelas de forma eficiente.

6. Contribución a los resultados (Outcomes)
Esta disciplina contribuye al logro de los siguientes resultados de la carrera:
1) S.O. Analizar un problema computacional complejo y aplicar los principios computacionales y otras disciplinas relevantes para identificar soluciones. (Usar) 6) S.O. Aplicar la teoría de la computación y los fundamentos del desarrollo de software para producir soluciones basadas en computación. (Usar)
7. Unidades

UNIDAD 1: Bases teóricas de la Computación Paralela y Distribuida Resultados del estudiante: 1,6	
Contenido	Objetivos Generales
▪ ¿Por qué y para qué Computación Paralela y Distribuida? ▪ Procesadores multinúcleo. ▪ Memoria compartida vs memoria distribuida. ▪ Multiprocesamiento simétrico. ▪ SIMD, procesamiento de vectores. ▪ GPU, coprocesamiento. ▪ Taxonomía de Flynn. ▪ Problemas de Memoria: • Caches multiprocesador y coherencia de caché. • Acceso a Memoria no uniforme (NUMA). ▪ Topologías: • Interconexiones. • Clusters. • Compartir recursos (p.e., buses e interconexiones). ▪ La ley de Amdahl: la parte de la computación que no se puede acelerar limita el efecto de las partes que sí pueden.	▪ Distinguir el uso de recursos computacionales para una respuesta más rápida para administrar el acceso eficiente a un recurso compartido [Familiarizarse] ▪ Explicar las diferencias entre memoria distribuida y memoria compartida [Evaluar] ▪ Describir la arquitectura SMP y observar sus principales características [Evaluar] ▪ Distinguir los tipos de tareas que son adecuadas para máquinas SIMD [Usar] ▪ Describir las ventajas y limitaciones de GPUs vs CPUs [Usar] ▪ Explicar las características de cada clasificación en la taxonomía de Flynn [Usar] ▪ Describir los desafíos para mantener la coherencia de la caché [Familiarizarse] ▪ Describir los desafíos clave del desempeño en diferentes memorias y topologías de sistemas distribuidos [Familiarizarse] ▪ Usar herramientas de software para perfilar y medir el desempeño de un programa [Usar]

Lecturas: Pacheco (2011), Schmidt, Kirk and Hwu (2013)

UNIDAD 2: Sistemas distribuidos Resultados del estudiante: 1,6

Contenido	Objetivos Generales
▪ Fallos: • Fallos basados en red (incluyendo particiones) y fallos basados en nodos • Impacto en garantías a nivel de sistema (p.e., disponibilidad) ▪ Envío de mensajes distribuido: • Conversión y transmisión de datos • Sockets • Secuenciamiento de mensajes • Almacenamiento, buffering, reenvío y descarte de mensajes ▪ Compensaciones de diseño para Sistemas Distribuidos: • Latencia versus rendimiento • Consistencia, disponibilidad, tolerancia de particiones ▪ Diseño de Servicio Distribuido: • Protocolos y servicios stateful versus stateless • Diseños de sesión (basados en la conexión) • Diseños reactivos (provocados por E/S) y diseños de múltiples hilos ▪ Algoritmos de Distribución de Núcleos: • Elección, descubrimiento	▪ Distinguir las fallas de red de otros tipos de fallas [Familiarizarse] ▪ Explicar por qué estructuras de sincronización como cerraduras simples (locks) no son útiles en la presencia de fallas distribuidas [Familiarizarse] ▪ Escribir un programa que realice cualquier proceso de marshalling requerido y la conversión en unidades de mensajes para comunicar datos importantes entre dos hosts [Usar] ▪ Medir el rendimiento observado y la latencia de la respuesta a través de los hosts en una red dada [Usar] ▪ Explicar por qué un sistema distribuido no puede ser simultáneamente Consistente (Consistent), Disponible (Available) y Tolerante a fallas (Partition tolerant) [Familiarizarse]

Lecturas: Pacheco (2011), **Schmidt**

UNIDAD 3: Comunicación y coordinación
Resultados del estudiante: 1,6

Contenido	Objetivos Generales
▪ Memoria Compartida. ▪ La consistencia, y su papel en los lenguajes de programación garantizando los programas libres de carreras. ▪ Pasos de Mensaje: • Mensajes Punto a Punto versus multicast (o basados en eventos) • Estrategias para recibir mensajes Blocking vs non-Blocking ▪ Buffers de mensaje ▪ Atomicidad: • Especificar y probar atomicidad y requisitos de granularidad • Requisitos de accesos atómicos y actualizaciones, y su impacto en la ejecución de procesos críticos o transacciones seguras ▪ Consistencia: • Condiciones de carrera, bloqueos, semáforos, etc. • Especificar causas de fallas y bloqueos (deadlocks) (causas, detección, prevención y evasión) • Condiciones de sincronización granulares más sofisticadas • Monitores y sincronización optimista/pesimista ▪ Consensos: • Elección, barreras, contadores y estructuras relacionadas ▪ Acciones condicionales: • Espera condicional (p.e., ejemplo variables de condición y señales) • La ley de Amdahl. ▪ Protocolos/coordinación y algoritmos paralelos segmentados. ▪ Efectos de algoritmos paralelos no-escalables.	▪ Usar exclusión mutua para evitar una condición de carrera [Usar] ▪ Dar un ejemplo de una ordenación de accesos entre actividades concurrentes (por ejemplo, un patrón de condición de carrera) que no son secuencialmente consistentes [Familiarizarse] ▪ Dar un ejemplo de un escenario en el que el bloqueo de mensajes impide que el programa proceda y explicar cómo esto se puede evitar usando mensajes no bloqueantes [Usar] ▪ Explicar cuándo se prefieren buffers atómicos frente a mensajes atómicos [Familiarizarse] ▪ Identificar condiciones de carrera comunes que impiden que dos procesos concurrentes hayan sido completados [Usar] ▪ Identificar fallas y vulnerabilidades causadas por condiciones de carrera [Familiarizarse] ▪ Definir: común término, trabajo y span [Familiarizarse] ▪ Comparar el diseño de una operación paralela con respecto a un algoritmo en ejecución secuencial [Usar] ▪ Explicar cómo un pipeline, una tubería de producción [Usar] ▪ Identificar los problemas que surgen en los algoritmos de tipo productor-consumidor y cómo los mecanismos pueden utilizarse para superar dichos problemas [Usar]

Lecturas: Pacheco (2011), Schmidt

UNIDAD 4: Programación para el Procesamiento Masivamente Paralelo
Resultados del estudiante: 1,6

Contenido	Objetivos Generales
▪ GPU, coprocesamiento. ▪ SIMD, procesamiento de vectores. ▪ Gestión dinámica de memoria, aproximaciones y técnicas: malloc/free, garbage collection (mark-sweep, copia, referencia), neighbors (también conocidos como arenas o zonas). ▪ Programación y contención. ▪ Consumo de energía y gestión.	▪ Explicar las características de cada clasificación en la taxonomía de Flynn [Usar] ▪ Describir las ventajas y limitaciones de GPUs vs CPUs [Usar] ▪ Comparar los beneficios de diferentes esquemas de administración de memoria, usando conceptos tales como Fragmentación, localidad y sobrecarga de memoria [Usar] ▪ Describir cómo la distribución/asignación de datos puede afectar a los costos de comunicación de un algoritmo [Familiarizarse] ▪ Explicar el impacto y los puntos de equilibrio relacionados al uso de energía en el desempeño paralelo [Familiarizarse]

Lecturas: Schmidt, Kirk and Hwu (2013)

8. Metodología

1. El profesor del curso presentará clases teóricas de los temas señalados en el programa propiciando la intervención de los alumnos.
2. El profesor del curso presentará demostraciones para fundamentar clases teóricas.
3. El profesor y los alumnos realizarán prácticas.
4. Los alumnos deberán asistir a clase habiendo leído lo que el profesor va a presentar. De esta manera se facilitará la comprensión y los estudiantes estarán en mejores condiciones de hacer consultas en clase.

9. Evaluar

- **Evaluación Permanente:** 60 %
 - **Evaluación Permanente 1:** 40 %
 - **Evaluación Permanente 2:** 60 %
- **Examen parcial:** 20 %
- **Examen final:** 20 %

Referencias

- Kirk, David B. and Wen-mei W. Hwu (2013). *Programming Massively Parallel Processors: A Hands-on Approach*. 2nd. Morgan Kaufmann. ISBN: 978-0-12-415992-1.
- Pacheco, Peter S. (2011). *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann. ISBN: 978-0-12-374260-5.